Reinforcement Learning
Lecture IX: Function Approximation

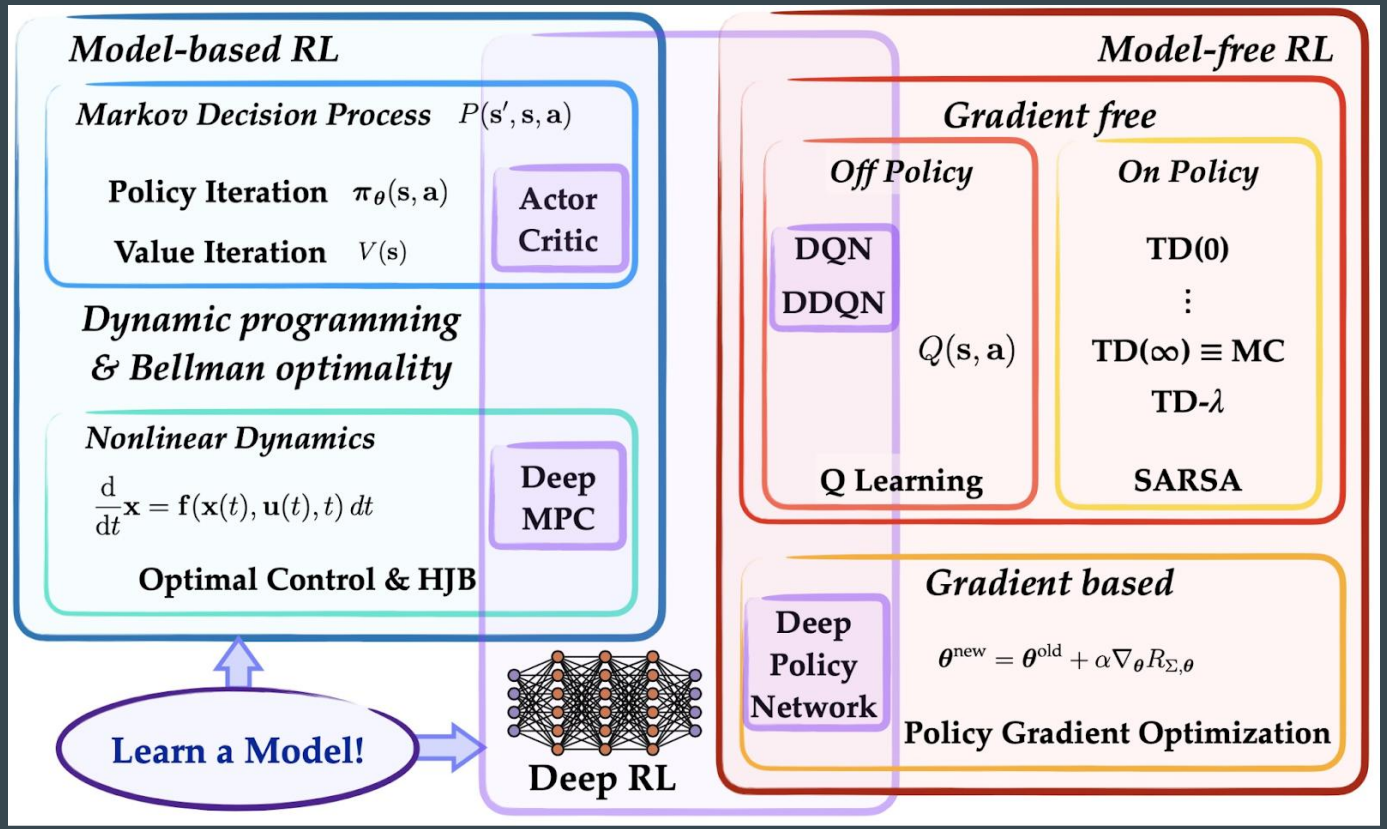
Asad Jeewa
University of Cape Town



What will be covered

- Function Approximation
- From Q-Learning to DQN

The Bigger Picture



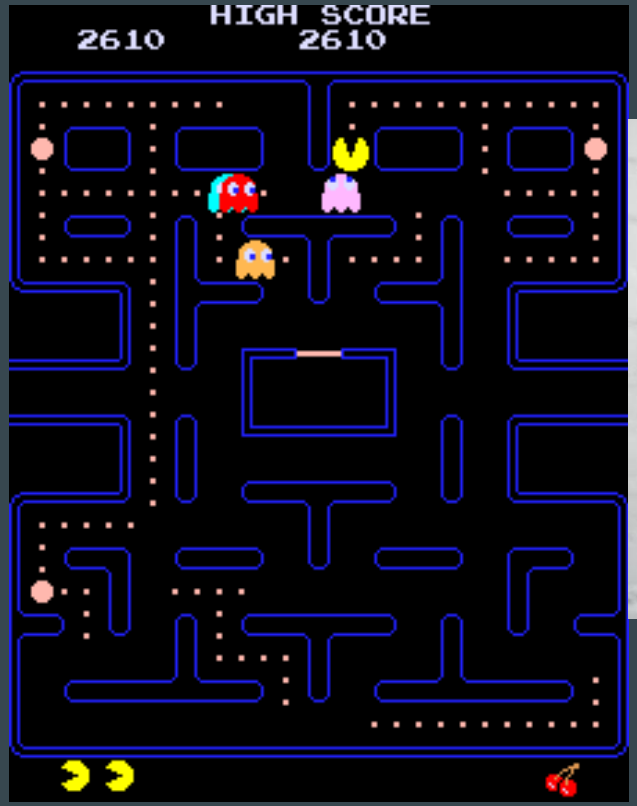
Recall: Q-Learning

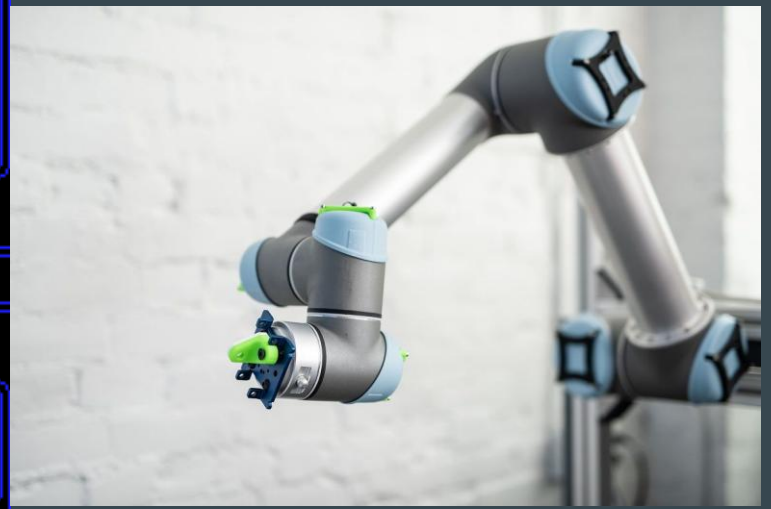
- **Goal:** Learn optimal action-value function $Q^* = (s, a)$

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s,a) \right]$$

- Model-free: No knowledge of transition dynamics needed
- Bootstrapping: Uses existing Q-values to update estimates
- **Table-based: Stores a value for all state action pairs**
- Off-policy: Learns optimal policy regardless of current behaviour

Do Tabular Methods Scale?

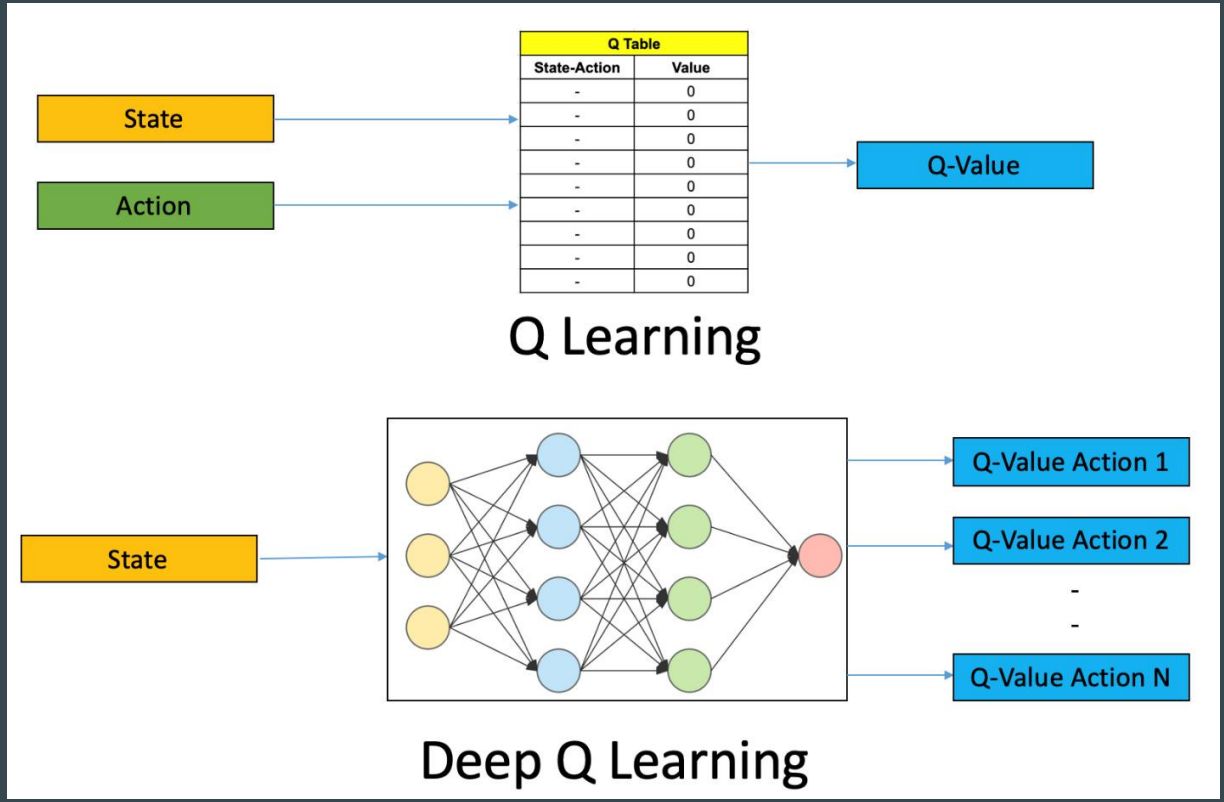




Do Tabular Methods Scale?

- Real-world tasks often have huge or continuous state spaces
- Memory requirements explode.
- Learning every state-action pair becomes impossible.
- If we never visit most states, how can we estimate their values?
- Generalisation across states is necessary.

Learning without a Table



https://thesequence.substack.com/p/edge95

Learning without a Table

- Function approximation replaces the Q-table.
- Learn parameters θ of a function $Q(s, a; \theta)$
- The function can generalise to unseen states.

Approximating the Value Function

- Represent $Q(s, a)$ using a parameterised function $Q(s, a; \theta)$
  - Linear models
  - Decision trees
  - **Neural networks (Non-Linear Models)**
- Learn via TD error minimisation:

$$\delta = r + \gamma \max_{a'} Q(s', a'; \theta) - Q(s, a; \theta)$$

https://medium.com/@samina.amin/deep-q-learning-dqn-71c109586bae

What is the parametrised function?

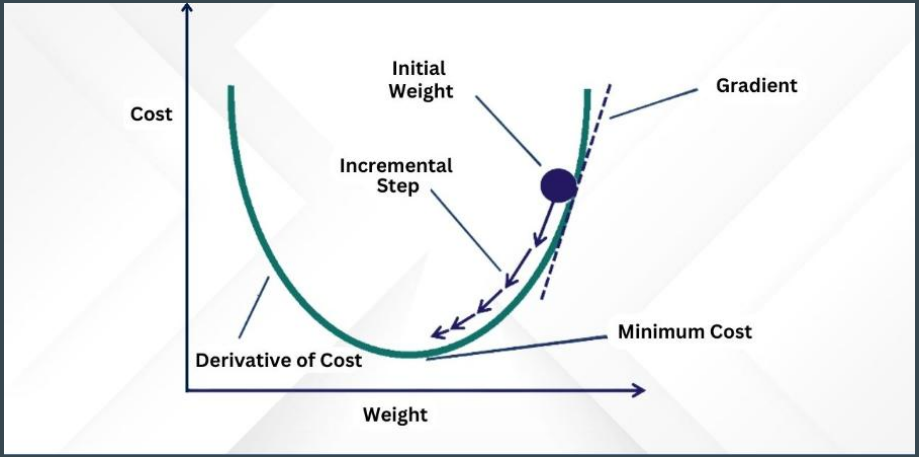
- “Black box” that maps inputs → outputs via adjustable parameters.
  - state-action → value
  - Internal workings will be studied later
- Learning = tuning parameters to improve predictions of expected rewards.
- As the function becomes more powerful, it can better generalise to unseen situations
  - Tabular RL: memorises exact values, no generalisation
  - Linear RL: simple linear model, generalises moderately
  - Deep RL: complex neural network, generalises strongly

Gradient Descent

- Goal: minimise a loss function $L(\theta)$; θ represents the model parameters

Update rule: $\theta \leftarrow \theta - \alpha \nabla_\theta L(\theta)$

- Moves parameters in direction of steepest decrease
- Used in RL to update network weights to reduce TD error



DQN

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
Initialize action-value function Q with random weights
for episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  end for
end for

epsilon-greedy exploration

step in the environment

replay buffer and sampling

regression target

gradient step

https://araffin.github.io/post/rl102/

DQN: Approximating the Value Function

- Model predicts y

$$y = r + \gamma a' \max_{a'} Q(s', a'; \theta)$$

- Learn by minimizing Mean Squared Error

$$L(\theta) = (y - Q(s, a; \theta))^2$$

- Optimise parameters θ via gradient descent

Limitations in DQN

- Correlated samples: Gradients are noisy and learning is unstable
- Non-stationary targets: The network keeps chasing moving Q-values
- Bootstrapping: Updates depend on the network's own predictions, so errors can reinforce themselves and destabilise learning.

Stabalising DQN

Two main techniques

- Experience Replay
  - Store transitions (s,a,r,s')
  - Sample randomly to break correlation and improve efficiency
- Target Network
  - Separate network for computing targets
  - Update slowly to stabilises learning

- Research paper: Human-level control through deep reinforcement learning

What about Gradient-based MC

Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$

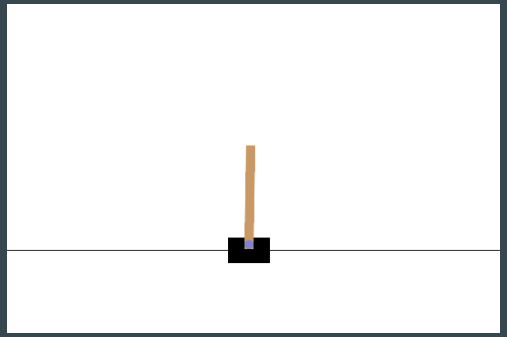
Input: the policy π to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \to \mathbb{R}$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, S_1, A_1, \ldots, R_T, S_T$ using π
  Loop for each step of episode, $t = 0, 1, \ldots, T-1$:
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha \big[G_t - \hat{v}(S_t, \mathbf{w}) \big] \nabla \hat{v}(S_t, \mathbf{w})$

- Update weights of a differentiable function
- Multiply by the gradient to adjust each weight in proportion to its effect on the prediction

PRACTICAL TASK

- Follow this simple tutorial to implement DQN on the seminal Cartpole problem.
- Extension: Adapt your code to train your DQN on a more complex environment such as an Atari Game





Resources

- Shared RL code space
- Sutton & Barto (1998), Reinforcement Learning: An Introduction, Chapter 9.
- https://huggingface.co/learn/deep-rl-course/en/unit3/deep-q-algorithm